APPENDIX F

# Further Reading

This book is a first course. Every chapter opens a door into a large field, and this appendix suggests where to go through it. The list is deliberately short and annotated: a few books and papers per topic, chosen because they are classics, because they are readable, or because they continue directly from where the book stops. Online resources are named rather than linked, since addresses change; a search for the title finds them.

**In this appendix you will learn:**

- where to continue with Lean and Mathlib;
- the standard textbooks on type theory, logic and category theory;
- the original papers behind the main ideas of the book;
- the research on Rust's type system, ownership and verification.

## F.1 Lean and Mathlib

- **Functional Programming in Lean**, by David Thrane Christiansen. The best introduction to Lean 4 as a programming language: structures, type classes, monads, `do` notation, and how Lean compiles. A natural continuation of Chapter 2.
- **Theorem Proving in Lean 4**, by Jeremy Avigad, Leonardo de Moura, Soonho Kong and Sebastian Ullrich. The reference introduction to Lean's logic: dependent types, inductive types, propositions, tactics, type classes, axioms.
- **Mathematics in Lean**, by Jeremy Avigad and Patrick Massot. How to do mathematics with Mathlib: the tactics, the library's conventions, and many worked exercises.
- **The Natural Number Game**, by Kevin Buzzard and others. A browser game that teaches tactic proofs by building the natural numbers from scratch; an enjoyable way to make the tactics of this book second nature.
- **The Mechanics of Proof**, by Heather Macbeth. An introduction to writing proofs, using Lean throughout; good if the proofs in this book were your first.
- The Mathlib documentation (searchable online), and Lean's own source code and Mathlib's, which your editor opens with a click on any name.
- Leonardo de Moura and Sebastian Ullrich, *The Lean 4 Theorem Prover and Programming Language* (CADE, 2021): the design of Lean 4.

- Sebastian Ullrich and Leonardo de Moura, *Counting Immutable Beans: Reference Counting Optimized for Purely Functional Programming* (2019), and Alex Reinking, Ningning Xie, Leonardo de Moura and Daan Leijen, *Perceus: Garbage Free Reference Counting with Reuse* (PLDI, 2021): how Lean updates unshared values in place (Chapter 24).

## F.2 Type theory and programming languages

- Benjamin C. Pierce, **Types and Programming Languages** (MIT Press, 2002). *The* textbook. Its first half covers, in more depth and with more variations, the ground of Chapters 3 and 7–13 of this book: untyped and typed λ-calculi, type safety, subtyping, recursive types, polymorphism, type reconstruction.
- Benjamin C. Pierce (editor), **Advanced Topics in Types and Programming Languages** (MIT Press, 2005). Its first chapter, David Walker's *Substructural Type Systems*, is the standard introduction to linear and affine type systems (Part V).
- Robert Harper, **Practical Foundations for Programming Languages** (Cambridge University Press, second edition 2016). A rigorous, unified treatment of programming languages through judgments and rules; the style of Chapter 3, taken all the way.
- **Software Foundations**, by Benjamin C. Pierce and many coauthors (online, in several volumes). Logic, programming-language semantics and type systems, formalized in the Rocq (formerly Coq) proof assistant; everything this book does in Lean for its toy languages, done systematically.
- **Programming Language Foundations in Agda**, by Philip Wadler, Wen Kokke and Jeremy G. Siek (online). The same ground in Agda, with intrinsically typed terms as in Chapter 8.
- Henk Barendregt, **The Lambda Calculus: Its Syntax and Semantics** (North-Holland, revised edition 1984). The encyclopedia of the untyped λ-calculus (Chapter 7).
- Per Martin-Löf, **Intuitionistic Type Theory** (Bibliopolis, 1984). The lectures that founded dependent type theory, of which Lean's is a descendant (Chapter 30).

## F.3 Logic and the Curry–Howard correspondence

- Jean-Yves Girard, Yves Lafont and Paul Taylor, **Proofs and Types** (Cambridge University Press, 1989). A short, dense classic: natural deduction, normalization, System F, and the beginnings of linear logic.
- Morten Heine Sørensen and Paweł Urzyczyn, **Lectures on the Curry–Howard Isomorphism** (Elsevier, 2006). The correspondence in all its variations, from intuitionistic logic to System F and dependent types.
- Philip Wadler, *Propositions as Types* (Communications of the ACM, 2015). An essay on the history and the meaning of the correspondence, and the best short read on the subject of Part II.
- Dag Prawitz, **Natural Deduction: A Proof-Theoretical Study** (1965), and Gerhard Gentzen's *Investigations into Logical Deduction* (1935), where natural deduction and the sequent calculus were born (Chapter 4).

- A. S. Troelstra and H. Schwichtenberg, **Basic Proof Theory** (Cambridge University Press, second edition 2000). Natural deduction, sequent calculi, normalization and cut elimination, for the reader who wants the full metatheory.
- Arthur Prior, *The Runabout Inference-Ticket* (1960): the paper that introduced `tonk` (Chapter 4).
- Frank Pfenning's lecture notes on constructive logic and on linear logic (from his courses at Carnegie Mellon, online): beautifully clear treatments of harmony, verifications and the judgmental approach that Chapters 4, 5 and 21 follow.
- Jean-Yves Girard, *Linear Logic* (Theoretical Computer Science, 1987): the paper that started it all (Chapter 21). Philip Wadler's *Linear Types Can Change the World!* (1990) is a much gentler introduction to linear types for programmers.

## F.4 Category theory

- Bartosz Milewski, **Category Theory for Programmers** (2019; also a series of blog posts and video lectures). The friendliest introduction for programmers, with examples in Haskell and C++; complements Part IV well.
- Benjamin C. Pierce, **Basic Category Theory for Computer Scientists** (MIT Press, 1991). Short and to the point.
- Steve Awodey, **Category Theory** (Oxford University Press, second edition 2010). A textbook written for readers coming from logic and computer science rather than from algebra.
- Emily Riehl, **Category Theory in Context** (Dover, 2016). A mathematician's introduction, with an outstanding treatment of the Yoneda lemma, limits and adjunctions.
- Saunders Mac Lane, **Categories for the Working Mathematician** (Springer, second edition 1998). The classic reference.
- J. Lambek and P. J. Scott, **Introduction to Higher Order Categorical Logic** (Cambridge University Press, 1986). Cartesian closed categories and the typed λ-calculus (Chapter 19).
- Eugenio Moggi, *Notions of Computation and Monads* (Information and Computation, 1991), and Philip Wadler, *Monads for Functional Programming* (1995): monads as a structure for effects (Chapter 20).

## F.5 Polymorphism, inference and abstraction

- John C. Reynolds, *Types, Abstraction and Parametric Polymorphism* (1983), and Philip Wadler, *Theorems for Free!* (1989): parametricity and free theorems (Chapter 18).
- Robin Milner, *A Theory of Type Polymorphism in Programming* (1978), and Luis Damas and Robin Milner, *Principal Type-Schemes for Functional Programs* (1982): Hindley–Milner type inference (Chapter 13).
- Philip Wadler and Stephen Blott, *How to Make Ad-hoc Polymorphism Less Ad Hoc* (1989): type classes (Chapter 14).

- John C. Mitchell and Gordon D. Plotkin, *Abstract Types Have Existential Type* (1988): existential types (Chapter 15).

## F.6 Rust, ownership and regions

- **The Rust Programming Language** ("the book"), **The Rust Reference** and **The Rustonomicon**, all published by the Rust project. The Reference describes the language precisely; the Rustonomicon explains `unsafe`, variance, drop check and the other dark corners that Chapters 24–27 and 32 touch.
- Mads Tofte and Jean-Pierre Talpin, *Region-Based Memory Management* (Information and Computation, 1997), and Dan Grossman and coauthors, *Region-Based Memory Management in Cyclone* (PLDI, 2002): the region calculi behind lifetimes (Chapter 26).
- Rust RFC 2094, *Non-lexical lifetimes*, and Niko Matsakis's blog *Baby Steps*, where NLL, Polonius and many other parts of the borrow checker were designed in public (Chapter 29).
- Aaron Weiss, Olek Gierczak, Daniel Patterson and Amal Ahmed, *Oxide: The Essence of Rust* (2019): a formal core calculus for Rust's ownership and borrowing, in the spirit of Chapter 28.
- Ralf Jung, Jacques-Henri Jourdan, Robbert Krebbers and Derek Dreyer, *RustBelt: Securing the Foundations of the Rust Programming Language* (POPL, 2018), and Ralf Jung's doctoral thesis *Understanding and Evolving the Rust Programming Language* (2020): semantic soundness of Rust and its standard library in the Iris separation logic (Chapter 32).
- Ralf Jung, Hoang-Hai Dang, Jeehoon Kang and Derek Dreyer, *Stacked Borrows: An Aliasing Model for Rust* (POPL, 2020), and its successor, Tree Borrows: the aliasing rules that Miri checks.
- John C. Reynolds, *Separation Logic: A Logic for Shared Mutable Data Structures* (LICS, 2002): the logic that RustBelt builds on.

## F.7 Verifying Rust programs

- Son Ho and Jonathan Protzenko, *Aeneas: Rust Verification by Functional Translation* (ICFP, 2022): translating safe Rust into pure functional code that can be verified in Lean, among other provers. The most direct continuation of Chapter 31.
- The documentation of **Verus**, **Creusot**, **Prusti** and **Kani**, tools that verify Rust programs with different techniques (specifications checked by SMT solvers, deductive verification, model checking). Read Chapter 31 for a short orientation, then each project's own guide.